Install guide

Signal Explorer | Windows source installation
Version 0.1.0 - prepared October 22, 2026

This guide installs the current Python desktop application on another PC. A standalone installer will be addressed later. You need internet access for the initial Python package downloads and any required driver downloads.

1. Prepare the PC and project files

Install **64-bit Python 3.12** and its Python launcher. This is the version used on the development PC. The project declares Python 3.11 or later, but other versions have not been validated here.

Copy or clone the project into a writable folder, for example **C:\Tools\Signal Explorer**. Include **pyproject.toml**, **README.md**, and the complete **src** folder. Copy **examples** for the demo DBC and **tests** if you want to run checks. Do not copy the old **.venv**; create one on the new PC. Git is optional if you copy the files.

2. Install the application

Open PowerShell in the project root - the folder containing pyproject.toml - and run:

```
cd "C:\Tools\Signal Explorer"
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install .
```

Using the virtual environment executable directly avoids the need to activate it or change PowerShell script-execution settings.

3. Launch Signal Explorer

```
.\.venv\Scripts\python.exe -m signal_explorer
```

Run this command from the project root whenever you want to start the application. The existing SignalExplorer.bat contains the development PC's absolute path; update that path before using it elsewhere.

Packages installed automatically

PySide6-Essentials - desktop UI; **python-can** - CAN interfaces; **pyserial** - serial adapters; **cantools** - DBC decoding; **uptime** - timestamp support. pip also installs their required dependencies. Build tooling includes setuptools. These are application dependencies, separate from vendor hardware drivers.

Adapter requirements

Install only the software needed for the adapters you plan to use.

Use vendor-supplied drivers and APIs matching this 64-bit installation. Driver installers may require Windows administrator approval. Vendor DLLs are not included in Signal Explorer's source files.

Virtual CAN - no hardware

No vendor software is required. Choose Virtual and enable Inject demo traffic to test the application without an adapter.

CANable - SLCAN firmware

Windows must recognize the adapter as a serial COM port. Install its appropriate USB serial driver if Windows does not configure it automatically. No additional Python packages are required. Choose CANable (SLCAN serial), select its actual COM port, and match the CAN bus bitrate. The new PC may assign a port other than COM3.

CANable - candleLight / gs_usb firmware

This is a different USB connection path from SLCAN. Install the additional backend:

```
.\.venv\Scripts\python.exe -m pip install "python-can[gs-usb]"
```

A compatible USB driver and libusb backend may also be needed on Windows. Configure those for the device's firmware, then choose CANable / candleLight (gs_usb). This path has not been physically validated in this project; the tested CANable connection is SLCAN.

Vector - VN16xx / CANXL adapter option

Install Vector hardware drivers **and the 64-bit XL Driver Library**. The USB driver alone is insufficient. Signal Explorer checks PATH and the standard Public Documents/Vector/XL Driver Library*/bin folder. If detection fails, use **Vector DLL** to select the installed **vxlapi64.dll**, then restart. Select the detected serial number and physical CAN channel; no application assignment in Vector Hardware Config is required.

Vector XL Driver Library

Kvaser - including U100-C

Install Kvaser Windows drivers with the CANlib runtime. Select Kvaser (U100 / CANlib), then Refresh devices. Channel indices start at zero. A listed Kvaser virtual channel is not a physical U100-C.

PEAK - including PCAN-USB Pro FD

Install PEAK hardware drivers and the 64-bit **PCAN-Basic API** (PCANBasic.dll). Select the PEAK option and a detected CAN channel. Manual names include PCAN_USBBUS1 and PCAN_USBBUS2; use detected names when several adapters are attached.

Intrepid - ValueCAN / neoVI

Install Intrepid hardware drivers and its native 64-bit **icsneo40.dll**, then install the optional Python dependencies:

```
.\.venv\Scripts\python.exe -m pip install ".[intrepid]"
```

This adds python-ics and filelock. Select the Intrepid adapter by serial number, not its COM port. For ValueCAN4-2, HSCAN is CAN1 and HSCAN2 is CAN2.

Verify and troubleshoot

A short check before connecting to a live CAN bus.

1. Verify the software without hardware

Launch Signal Explorer, select **Virtual**, enable **Inject demo traffic**, and click **Connect**. The trace should show ID 100 with changing data. Load **examples/demo.dbc** to display the demo message name and decoded signals.

Auto-scroll (all messages) is checked by default and shows timestamped history. Uncheck it for the latest data per channel and ID. History retains the most recent 10,000 frames. **Pause Exploring** freezes trace updates while staying connected; **Start Exploring** resumes. Frames received while paused are not replayed.

2. Verify hardware reception

Connect the adapter by USB, choose its backend and channel, set the correct bus bitrate, and click Connect. Use a powered, correctly wired and terminated CAN bus with a device that is transmitting. Check the RX count. Click Clear filter if frames might be hidden. Merely connecting USB does not generate CAN traffic.

Common issues

Python or py is not recognized: verify Python 3.12 is installed and reopen PowerShell. Check with **py -3.12 --version**.

No module named signal_explorer: run the install command from the project root and launch using the same .venv Python executable.

COM port access denied: disconnect that port in other Signal Explorer windows and close other programs using it. Only one process can hold the CANable serial port.

Connected, RX stays at zero: confirm the bus has traffic, the bitrate matches, CAN-H/CAN-L wiring is correct, and the required termination is present. A quiet bus can connect successfully without producing trace rows.

RX increases but the trace appears empty: click Clear filter and ensure exploring is running. While paused, trace counters and rows stay frozen.

Missing DLL or no detected channels: install the vendor's 64-bit API as well as its USB driver. Reconnect USB, use Refresh devices, or restart after changing DLL settings. No attached device can legitimately produce an empty channel list.

Optional automated checks

If the tests folder was copied, run the following from the project root. The complete suite also expects the Intrepid optional dependencies to be installed.

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Tests use simulated or virtual traffic and do not establish that a new physical adapter is wired and communicating correctly.

Current scope

The app supports **Classic CAN and CAN FD, one channel at a time**. Enable CAN FD and set arbitration/data bitrates before connecting. Virtual FD demo traffic includes Engine RPM, MAF and O2 signals; load examples/demo.dbc to decode them. Physical FD operation still needs hardware validation. CAN XL, LIN, Ethernet and SENT are not yet exposed. The Vector dropdown's CANXL label does not enable CAN XL frames.

DBC files are optional; copy the files you intend to use separately. No compiler is normally needed when compatible prebuilt Python wheels are available. Record working package versions for reproducible deployments; the current project uses minimum version constraints rather than a pinned dependency lock.